

Tugas Mandiri 12: Laporan Praktikum Mandiri 12 Machine Learning

Maryam Hasnaa' Syamila - 0110222067

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

E-mail: hasyamil4@gmail.com

Link Notebook Colab:

 Maryam Hasnaa' Syamila_0110222067_PraktikumMandiri12.ipynb

Link GitHub Tugas:

<https://github.com/maryamhasnaasyamila/machine-learning/tree/tugas/praktikum/praktikum12>

Link GitHub Praktikum:

<https://github.com/maryamhasnaasyamila/machine-learning/tree/praktikum/praktikum/praktikum12>

Link Dataset:

[Breast Cancer Classification Using PCA in Python](#)

Analisis Perbandingan Model SVM Tanpa PCA dan Dengan PCA pada Klasifikasi Data Breast Cancer

1. Import Library

```
1  import numpy as np
2  import pandas as pd
3
4  # Library untuk preprocessing dan model
5  from sklearn.model_selection import train_test_split
6  from sklearn.preprocessing import StandardScaler
7  from sklearn.decomposition import PCA
8  from sklearn.svm import SVC
9
10 # Library evaluasi model
11 from sklearn.metrics import accuracy_score, classification_report
12
13 # Library visualisasi
14 import matplotlib.pyplot as plt
15 from mpl_toolkits.mplot3d import Axes3D
16
17 # Mengatur ukuran default grafik
18 plt.rcParams['figure.figsize'] = (8, 6)
```

Penjelasan Kode:

- Mengimpor library utama seperti **pandas** dan **numpy** untuk pengolahan data.
- Mengimpor modul dari **scikit-learn** untuk preprocessing data, PCA, dan model SVM.

- Mengimpor library visualisasi seperti `matplotlib` untuk menampilkan grafik.

Penjelasan Output:

- Tidak menghasilkan output visual.
- Menandakan seluruh library yang dibutuhkan telah siap digunakan.

2. Load Dataset

```
1 # sinkronisasi dengan google drive
2 from google.colab import drive
3 drive.mount('/content/drive')
```

Mounted at /content/drive

```
1 # path dataset di Google Drive
2 file_path = "/content/drive/My Drive/SEMESTER 7/Machine Learning/
praktikum12/datasets/data.csv"
3
4 # membaca dataset
5 df = pd.read_csv(file_path)
6
7 # menampilkan 5 baris pertama
8 df.head()
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness
0	842302	M	17.99	10.38	122.80	1001.0	
1	842517	M	20.57	17.77	132.90	1326.0	
2	84300903	M	19.69	21.25	130.00	1203.0	
3	84348301	M	11.42	20.38	77.58	386.1	
4	84358402	M	20.29	14.34	135.10	1297.0	

5 rows × 33 columns

Penjelasan Kode:

- Membaca dataset dari file CSV menggunakan `pd.read_csv`.
- Dataset dimuat ke dalam bentuk DataFrame agar mudah dianalisis.

Penjelasan Output:

- Menampilkan beberapa baris awal dataset.
- Memberikan gambaran awal mengenai struktur dan isi data.

3. Informasi Umum Dataset

```
1 # informasi umum dataset
2 df.info()

*** <class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   id                                         569 non-null    int64
1   diagnosis                                 569 non-null    object
2   radius_mean                              569 non-null    float64
3   texture_mean                             569 non-null    float64
4   perimeter_mean                           569 non-null    float64
5   area_mean                                569 non-null    float64
6   smoothness_mean                          569 non-null    float64
7   compactness_mean                         569 non-null    float64
8   concavity_mean                           569 non-null    float64
9   concave points_mean                      569 non-null    float64
10  symmetry_mean                            569 non-null    float64
11  fractal_dimension_mean                   569 non-null    float64
12  radius_se                                569 non-null    float64
13  texture_se                               569 non-null    float64
14  perimeter_se                             569 non-null    float64
15  area_se                                  569 non-null    float64
16  smoothness_se                           569 non-null    float64
17  compactness_se                           569 non-null    float64
18  concavity_se                             569 non-null    float64
19  concave points_se                        569 non-null    float64
20  symmetry_se                              569 non-null    float64
21  fractal_dimension_se                     569 non-null    float64
22  radius_worst                             569 non-null    float64
23  texture_worst                            569 non-null    float64
24  perimeter_worst                          569 non-null    float64
25  area_worst                               569 non-null    float64
26  smoothness_worst                         569 non-null    float64
27  compactness_worst                        569 non-null    float64
28  concavity_worst                          569 non-null    float64
29  concave points_worst                     569 non-null    float64
30  symmetry_worst                           569 non-null    float64
31  fractal_dimension_worst                  569 non-null    float64
32  Unnamed: 32                              0 non-null      float64
dtypes: float64(31), int64(1), object(1)
memory usage: 146.8+ KB
```

Penjelasan Kode:

- Menggunakan `df.info()` untuk melihat jumlah data, nama kolom, dan tipe data.
- Digunakan untuk memastikan dataset terbaca dengan benar.

Penjelasan Output:

- Menampilkan jumlah baris dan kolom dataset.
- Menunjukkan tipe data setiap fitur serta keberadaan nilai non-null.

4. Statistik Deskriptif

```
1 # statistik deskriptif
2 df.describe()
```

	id	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean
count	5.690000e+02	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000
mean	3.037183e+07	14.127292	19.289649	91.969033	654.889104	0.096360	0.104340
std	1.250206e+08	3.524049	4.301036	24.298981	351.914129	0.014064	0.052810
min	8.670000e+03	6.981000	9.710000	43.790000	143.500000	0.052630	0.019380
25%	8.692180e+05	11.700000	16.170000	75.170000	420.300000	0.086370	0.064920
50%	9.060240e+05	13.370000	18.840000	86.240000	551.100000	0.095870	0.092630
75%	8.813129e+06	15.780000	21.800000	104.100000	782.700000	0.105300	0.130400
max	9.113205e+08	28.110000	39.280000	188.500000	2501.000000	0.163400	0.345400

8 rows × 8 columns

Penjelasan Kode:

- Menggunakan `df.describe()` untuk menghitung statistik dasar.
- Statistik meliputi mean, standar deviasi, nilai minimum, dan maksimum.

Penjelasan Output:

- Menampilkan ringkasan statistik tiap fitur numerik.
- Membantu memahami sebaran dan skala data.

5. Cek Missing Value

```
1 # cek missing value
2 df.isnull().sum()
```

	0
id	0
diagnosis	0
radius_mean	0
texture_mean	0
perimeter_mean	0
area_mean	0
smoothness_mean	0
compactness_mean	0
concavity_mean	0
concave points_mean	0
symmetry_mean	0
fractal_dimension_mean	0
radius_se	0
texture_se	0
perimeter_se	0
area_se	0
smoothness_se	0
compactness_se	0
concavity_se	0
concave points_se	0
symmetry_se	0
fractal_dimension_se	0
radius_worst	0

Penjelasan Kode:

- Menggunakan `df.isnull().sum()` untuk mendeteksi nilai kosong.
- Bertujuan memastikan kualitas data sebelum diproses lebih lanjut.

Penjelasan Output:

- Menampilkan jumlah missing value pada setiap kolom.
- Teridentifikasi adanya kolom yang mengandung nilai NaN.

6. Cek Duplikasi Data

```
1 # cek data duplikat
2 df.duplicated().sum()

np.int64(0)
```

Penjelasan Kode:

- Menggunakan `df.duplicated().sum()` untuk mengecek data ganda.
- Bertujuan mencegah bias akibat duplikasi data.

Penjelasan Output:

- Menunjukkan jumlah data duplikat.
- Jika bernilai 0, berarti tidak ada data yang terduplikasi.

7. Pemisahan Fitur dan Label

```
1 # Menghapus kolom yang tidak digunakan sebagai fitur
2 # 'id' dihapus jika ada, dan 'diagnosis' sebagai target
3 X = df.drop(columns=['diagnosis', 'id'], errors='ignore')
4
5 # Label / target
6 y = df['diagnosis']
7
8 print("Shape fitur (X):", X.shape)
9 print("Shape label (y):", y.shape)

Shape fitur (X): (569, 31)
Shape label (y): (569,)
```

Penjelasan Kode:

- Memisahkan fitur (X) dan label/target (y).
- Kolom target tidak disertakan dalam fitur.

Penjelasan Output:

- Menampilkan bentuk (shape) data fitur dan label.
- Memastikan pemisahan data berjalan dengan benar.

8. Splitting Data (train & test)

```
1  # Membagi data 80% training dan 20% testing
2  X_train, X_test, y_train, y_test = train_test_split(
3      X,
4      y,
5      test_size=0.2,
6      random_state=42,
7      stratify=y # menjaga proporsi kelas
8  )
9
10 print("X_train:", X_train.shape)
11 print("X_test :", X_test.shape)
```

```
X_train: (455, 31)
X_test : (114, 31)
```

Penjelasan Kode:

- Membagi data menjadi data latih dan data uji menggunakan `train_test_split`.
- Proporsi umum yang digunakan adalah 80% data latih dan 20% data uji.

Penjelasan Output:

- Menampilkan jumlah data training dan testing.
- Menjamin model diuji pada data yang belum pernah dilihat.

9. Standardisasi Data

```
1 # Membuat objek StandardScaler
2 scaler = StandardScaler()
3
4 # Fit dan transform hanya pada data training
5 X_train_scaled = scaler.fit_transform(X_train)
6
7 # Transform data testing menggunakan scaler yang sama
8 X_test_scaled = scaler.transform(X_test)
9
10 # Menampilkan contoh hasil standardisasi
11 X_train_scaled[:5]
```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/extmath.py:1101: RuntimeWarning: invalid value encountered in divide
updated_mean = (last_sum + new_sum) / updated_sample_count
/usr/local/lib/python3.12/dist-packages/sklearn/utils/extmath.py:1106: RuntimeWarning: invalid value encountered in divide
T = new_sum / new_sample_count
/usr/local/lib/python3.12/dist-packages/sklearn/utils/extmath.py:1126: RuntimeWarning: invalid value encountered in divide
new_unnormalized_variance -= correction**2 / new_sample_count
array([[5.18558727e-01, 8.91825791e-01, 4.24631702e-01,
 3.83925436e-01, -9.74743706e-01, -6.89771505e-01,
 -6.88586446e-01, -3.98175254e-01, -1.03915470e+00,
 -8.25056321e-01, -1.09317755e-01, -5.59755400e-02,
 -2.10096206e-01, -1.59132582e-02, -1.00518399e+00,
 -9.11941990e-01, -6.62815884e-01, -6.52561081e-01,
 -7.01889114e-01, -2.75393571e-01, 5.79797697e-01,
 1.31324246e+00, 4.66908134e-01, 4.45982711e-01,
 -5.96154777e-01, -6.34722227e-01, -6.10227299e-01,
 -2.35743918e-01, 5.45663235e-02, 2.18367276e-02,
 nan],
 [-5.16364088e-01, -1.63971029e+00, -5.41348716e-01,
 -5.42961327e-01, 4.76219058e-01, -6.31833818e-01,
 -6.04281166e-01, -3.03074908e-01, 5.21543093e-01,
 -4.54522896e-01, -6.04377961e-01, -1.00104604e+00,
 -5.85429002e-01, -4.93453793e-01, 4.03212009e-01,
 -7.68173276e-01, -4.79187222e-01, 1.14508478e-01,
 -1.42950761e-01, -5.77397732e-01, -5.82458953e-01,
 -1.69029101e+00, -6.11934288e-01, -5.87013537e-01,
 2.73581959e-01, -8.14844486e-01, -7.12666415e-01,
 -3.23207881e-01, -1.37576237e-01, -9.04401643e-01,
 nan],
 [-3.68118388e-01, 4.55514964e-01, -3.88249933e-01,
 -4.02970039e-01, -1.43297929e+00, -3.83927370e-01,
 -3.42175070e-01, -7.65459348e-01, -8.50857052e-01,
 -2.26170901e-01, 3.03979894e-01, 1.05150064e+00,
 -1.69545176e-01, -8.09073190e-04, -3.10104091e-01,

Penjelasan Kode:

- Menggunakan `StandardScaler` untuk menyamakan skala fitur.
- Setiap fitur diubah agar memiliki mean 0 dan standar deviasi 1.

Penjelasan Output:

- Menampilkan data hasil standardisasi.
- Nilai fitur menjadi sebanding dan siap digunakan oleh model.

10. Cek Jumlah NaN di setiap kolom

```
1 # Cek jumlah NaN di setiap kolom
2 X.isnull().sum()
```

...	0
radius_mean	0
texture_mean	0
perimeter_mean	0
area_mean	0
smoothness_mean	0
compactness_mean	0
concavity_mean	0
concave points_mean	0
symmetry_mean	0
fractal_dimension_mean	0
radius_se	0
texture_se	0
perimeter_se	0
area_se	0
smoothness_se	0
compactness_se	0
concavity_se	0
concave points_se	0
symmetry_se	0
fractal_dimension_se	0
radius_worst	0
texture_worst	0
perimeter_worst	0

Penjelasan Kode:

- Mengecek kembali keberadaan nilai NaN setelah preprocessing awal.
- Digunakan untuk mencegah error pada model SVM.

Penjelasan Output:

- Menunjukkan masih adanya nilai NaN pada kolom tertentu.
- Menjadi dasar untuk proses pembersihan data selanjutnya.

11. Hapus Kolom yang mengandung NaN dan Unnamed

```
1 # Menghapus kolom tidak berguna yang seluruhnya NaN
2 df = df.drop(columns=['Unnamed: 32'], errors='ignore')
```



```
1 # Menghapus semua kolom yang namanya mengandung 'Unnamed'
2 df = df.loc[:, ~df.columns.str.contains('^Unnamed')]
```

Penjelasan Kode:

- Menghapus kolom yang seluruh nilainya kosong (NaN).
- Menghapus kolom **Unnamed** yang tidak memiliki makna fitur.

Penjelasan Output:

- Dataset menjadi lebih bersih.
- Kolom yang tidak relevan tidak lagi muncul.

12. Memastikan X sudah bersih dari NaN

```
1 # Cek Data yang Masuk ke Model (X)
2 X = df.drop(columns=['diagnosis', 'id'], errors='ignore')
```



```
1 # Pastikan X sudah bersih dari NaN
2 X.isnull().sum()
```

...

radius_mean	0
texture_mean	0
perimeter_mean	0
area_mean	0
smoothness_mean	0
compactness_mean	0
concavity_mean	0
concave points_mean	0
symmetry_mean	0
fractal_dimension_mean	0
radius_se	0
texture_se	0
perimeter_se	0
area_se	0
smoothness_se	0
compactness_se	0
concavity_se	0
concave points_se	0
symmetry_se	0
fractal_dimension_se	0
radius_worst	0

Penjelasan Kode:

- Mengecek ulang fitur (X) untuk memastikan tidak ada nilai NaN.
- Langkah ini penting sebelum masuk ke proses modeling.

Penjelasan Output:

- Menunjukkan jumlah NaN bernilai 0.
- Menandakan data siap digunakan untuk pelatihan model.

13. Pemisahan Ulang Fitur dan Label

```
1 X = df.drop(columns=['diagnosis', 'id'], errors='ignore')
2 y = df['diagnosis']
3
4 X.isnull().sum()
```

...

radius_mean	0
texture_mean	0
perimeter_mean	0
area_mean	0
smoothness_mean	0
compactness_mean	0
concavity_mean	0
concave points_mean	0
symmetry_mean	0
fractal_dimension_mean	0
radius_se	0
texture_se	0
perimeter_se	0
area_se	0
smoothness_se	0
compactness_se	0
concavity_se	0
concave points_se	0
symmetry_se	0
fractal_dimension_se	0
radius_worst	0
texture_worst	0

Penjelasan Kode:

- Melakukan pemisahan ulang X dan y setelah pembersihan data.
- Bertujuan memastikan data yang digunakan benar-benar bersih.

Penjelasan Output:

- Menampilkan struktur fitur dan label yang telah diperbarui.
- Tidak ditemukan kolom tidak relevan.

14. Splitting Data (ulang)

```
1 from sklearn.model_selection import train_test_split
2
3 X_train, X_test, y_train, y_test = train_test_split(
4     X,
5     y,
6     test_size=0.2,
7     random_state=42,
8     stratify=y
9 )
```

Penjelasan Kode:

- Membagi ulang data menjadi train dan test.
- Dilakukan agar data hasil cleaning digunakan sepenuhnya.

Penjelasan Output:

- Menampilkan ukuran data training dan testing terbaru.
- Menjamin konsistensi data setelah preprocessing ulang.

15. Standarisasi Data ulang

```
1 from sklearn.preprocessing import StandardScaler
2
3 scaler = StandardScaler()
4
5 X_train_scaled = scaler.fit_transform(X_train)
6 X_test_scaled = scaler.transform(X_test)
```

```
1 import numpy as np
2 np.isnan(X_train_scaled).sum()
```

```
np.int64(0)
```

Penjelasan Kode:

- Melakukan standarisasi ulang menggunakan data yang sudah bersih.
- Menghindari pengaruh data lama yang masih mengandung NaN.

Penjelasan Output:

- Data hasil scaling tidak mengandung nilai NaN.
- Siap digunakan untuk pelatihan SVM.

16. Model SVM tanpa PCA

```
1 from sklearn.svm import SVC
2 from sklearn.metrics import accuracy_score, classification_report
3
4 svm_no_pca = SVC(kernel='rbf', gamma='scale', random_state=42)
5 svm_no_pca.fit(X_train_scaled, y_train)
6
7 y_pred_no_pca = svm_no_pca.predict(X_test_scaled)
8
9 acc_no_pca = accuracy_score(y_test, y_pred_no_pca)
10
11 print("Akurasi SVM tanpa PCA:", acc_no_pca)
12 print("\nClassification Report (Tanpa PCA)")
13 print(classification_report(y_test, y_pred_no_pca))
```

Akurasi SVM tanpa PCA: 0.9736842105263158

Classification Report (Tanpa PCA)					
	precision	recall	f1-score	support	
B	0.96	1.00	0.98	72	
M	1.00	0.93	0.96	42	
accuracy			0.97	114	
macro avg	0.98	0.96	0.97	114	
weighted avg	0.97	0.97	0.97	114	

Penjelasan Kode:

- Membuat model SVM dengan kernel RBF sebagai baseline.
- Model dilatih menggunakan seluruh fitur asli.

Penjelasan Output:

- Menampilkan nilai akurasi model tanpa PCA.
- Classification report menunjukkan performa model pada setiap kelas.

17. Penerapan PCA

```
1  from sklearn.decomposition import PCA
2
3  # Membuat PCA dengan 3 komponen utama
4  pca = PCA(n_components=3)
5
6  # Fit PCA hanya pada data training
7  X_train_pca = pca.fit_transform(X_train_scaled)
8
9  # Transform data testing
10 X_test_pca = pca.transform(X_test_scaled)
11
12 print("Shape X_train_pca:", X_train_pca.shape)
13 print("Shape X_test_pca :", X_test_pca.shape)
14
15 # Variansi yang dijelaskan
16 explained_var = pca.explained_variance_ratio_
17 print("Explained Variance Ratio:", explained_var)
18 print("Total Variansi:", explained_var.sum())
19
```

*** Shape X_train_pca: (455, 3)
Shape X_test_pca : (114, 3)
Explained Variance Ratio: [0.44593522 0.18545255 0.09584641]
Total Variansi: 0.7272341886241546

Penjelasan Kode:

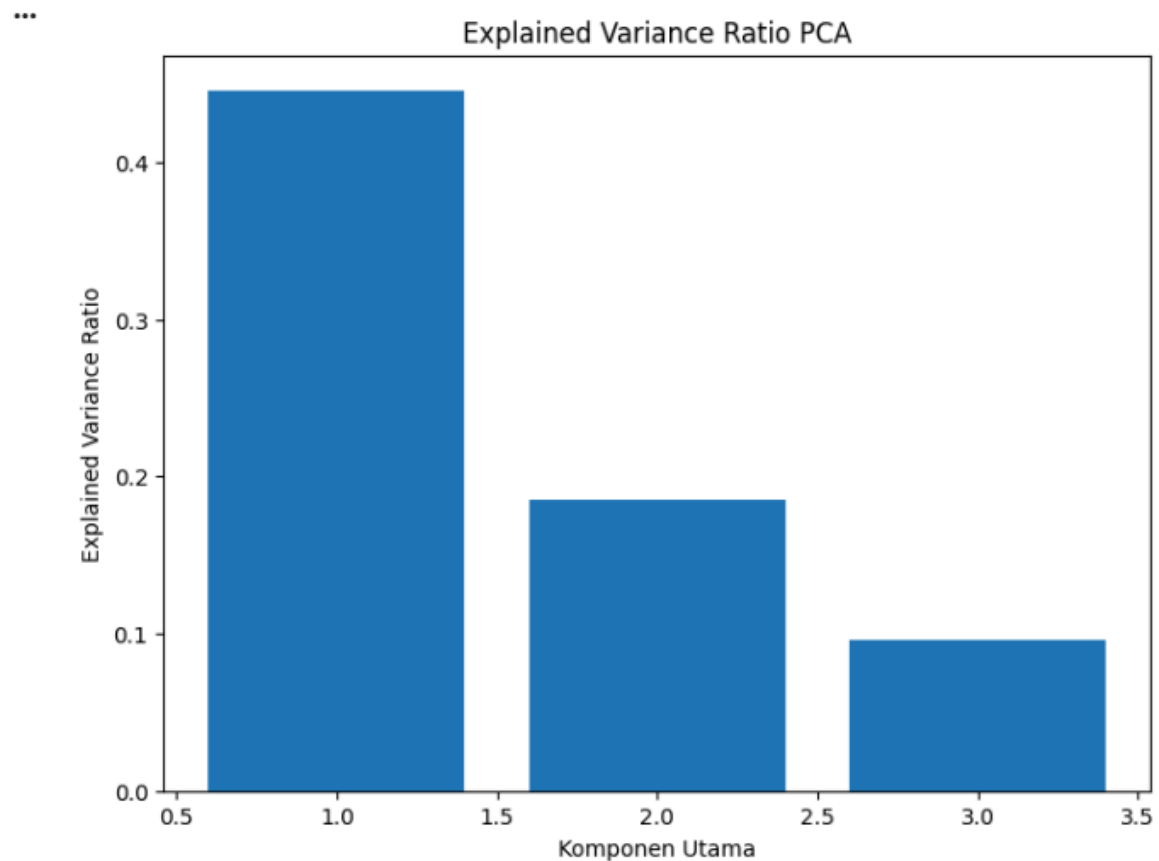
- Menerapkan PCA untuk mereduksi dimensi data menjadi 3 komponen utama.
- PCA dilakukan setelah data distandarisasi.

Penjelasan Output:

- Menampilkan bentuk data setelah reduksi dimensi.
- Menunjukkan proporsi variansi yang dijelaskan oleh tiap komponen.

18. Visualisasi Explained Variance

```
1 import matplotlib.pyplot as plt
2
3 plt.bar([1, 2, 3], explained_var)
4 plt.xlabel('Komponen Utama')
5 plt.ylabel('Explained Variance Ratio')
6 plt.title('Explained Variance Ratio PCA')
7 plt.show()
```



Penjelasan Kode:

- Membuat grafik batang explained variance ratio.
- Digunakan untuk melihat kontribusi setiap komponen PCA.

Penjelasan Output:

- Grafik menunjukkan sebagian besar variansi dijelaskan oleh komponen awal.
- Membantu menentukan jumlah komponen yang optimal.

19. SVM dengan PCA

```
1 svm_pca = SVC(kernel='rbf', gamma='scale', random_state=42)
2 svm_pca.fit(X_train_pca, y_train)
3
4 y_pred_pca = svm_pca.predict(X_test_pca)
5
6 acc_pca = accuracy_score(y_test, y_pred_pca)
7
8 print("Akurasi SVM dengan PCA:", acc_pca)
9 print("\nClassification Report (Dengan PCA)")
10 print(classification_report(y_test, y_pred_pca))
```

```
*** Akurasi SVM dengan PCA: 0.956140350877193
```

```
Classification Report (Dengan PCA)
```

	precision	recall	f1-score	support
B	0.94	1.00	0.97	72
M	1.00	0.88	0.94	42
accuracy			0.96	114
macro avg	0.97	0.94	0.95	114
weighted avg	0.96	0.96	0.96	114

Penjelasan Kode:

- Melatih kembali model SVM menggunakan data hasil PCA.
- Bertujuan membandingkan performa sebelum dan sesudah PCA.

Penjelasan Output:

- Menampilkan akurasi SVM dengan PCA.
- Classification report menunjukkan performa model setelah reduksi dimensi.

20. Kesimpulan Perbandingan

```
1 import pandas as pd
2
3 comparison = pd.DataFrame({
4     'Model': ['SVM tanpa PCA', 'SVM dengan PCA'],
5     'Jumlah Fitur': [X_train.shape[1], X_train_pca.shape[1]],
6     'Akurasi': [acc_no_pca, acc_pca],
7     'Total Variansi PCA': [None, explained_var.sum()]
8 })
9
10 comparison
```

	Model	Jumlah Fitur	Akurasi	Total Variansi PCA
0	SVM tanpa PCA	30	0.973684	NaN
1	SVM dengan PCA	3	0.956140	0.727234

Penjelasan Kode:

- Menyusun tabel perbandingan antara SVM tanpa PCA dan dengan PCA.
- Perbandingan meliputi jumlah fitur, akurasi, dan variansi PCA.

Penjelasan Output:

- Menunjukkan bahwa PCA berhasil mengurangi jumlah fitur secara signifikan.
- Akurasi sedikit menurun namun model menjadi lebih efisien.

21. Kesimpulan Akhir

Berdasarkan hasil pengujian, model SVM tanpa PCA menggunakan 30 fitur asli dan menghasilkan akurasi sebesar 97.36%. Setelah diterapkan PCA, jumlah fitur berhasil direduksi menjadi 3 komponen utama yang mampu mempertahankan sekitar 72.72% variansi data. Model SVM dengan PCA menghasilkan akurasi sebesar 95.61%. Meskipun terjadi sedikit penurunan akurasi, penggunaan PCA membuat model menjadi lebih efisien dengan kompleksitas yang jauh lebih rendah dan tetap mempertahankan performa klasifikasi yang baik.